

**Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»**

**Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий**

**ОТЧЁТ
по лабораторной работе № 1 (часть 2)
по дисциплине
«Проектирование защищенных интеллектуальных
информационных систем»**

Тема: Управление доступом к СУБД PostgreSQL

Выполнил:
студент группы 321701
Струнец Д. П.

Проверил:
Сальников Д. А.

Минск 2026

1 ПОСТАНОВКА ЗАДАЧИ

Целью работы является настройка и проверка политики контроля доступа выбранной СУБД. Согласно заданию необходимо создать программы для: установки СУБД; настройки ролей доступа; проверки политики; удаления созданных объектов.

Требуемая политика безопасности:

- администратор — полный доступ ко всем подсистемам;
- пользователь — полные либо частичные права в предоставленной подсистеме;
- гость — только чтение отдельных фрагментов, разрешённых администратором.

В качестве варианта выбрана реляционная СУБД PostgreSQL, разворачиваемая в контейнере Docker. Автоматизация выполнена на языке Python.

2 КРАТКОЕ ОПИСАНИЕ ВЫБРАННОЙ СУБД

PostgreSQL — объектно-реляционная СУБД с развитой моделью ролей и прав (GRANT/REVOKE) на уровне баз, схем, таблиц и последовательностей. Разграничение доступа реализуется внутри СУБД независимо от прав ОС: субъект (роль) получает ровно те операции, которые явно выданы. В работе использован образ postgres:16-alpine, порт 5432, суперпользователь контейнера — postgres.

3 ОПИСАНИЕ СОЗДАННЫХ ПРОГРАММ

3.1 Состав программ

- docker-compose.yml — установка/запуск PostgreSQL (контейнер psiiz-postgres);
- configure.py — создание ролей lab_admin / lab_user / lab_guest, БД psiiz_demo, таблиц и GRANT;
- test.py — проверка разрешённых и запрещённых операций для каждой роли;
- cleanup.py — удаление БД, ролей и служебного состояния lab_state.json;

– db.py — общие параметры подключения.

3.2 Настроенная политика

lab_admin создаётся с атрибутами SUPERUSER/CREATEDB/CREATEROLE и является владельцем базы — полный доступ. lab_user получает SELECT/INSERT/UPDATE/DELETE на public_data и только SELECT на confidential_data. lab_guest получает только SELECT на public_data; доступ к confidential_data запрещён.

Фрагмент назначения прав в configure.py:

```
run_sql(conn, "GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO lab_admin")
run_sql(conn, "GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO
lab_admin")

# user — частичные права: всё на public, только чтение confidential
run_sql(conn, "GRANT SELECT, INSERT, UPDATE, DELETE ON public_data TO lab_user")
run_sql(conn, "GRANT USAGE, SELECT ON SEQUENCE public_data_id_seq TO lab_user")
run_sql(conn, "GRANT SELECT ON confidential_data TO lab_user")

# guest — только чтение public_data
run_sql(conn, "GRANT SELECT ON public_data TO lab_guest")

run_sql(conn, "ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES
TO lab_guest")
```

3.3 Результаты проверки

Программа test.py подключается от имени каждой роли и фиксирует успех или отказ (permission denied). Все ожидаемые разрешения и запреты подтверждены (см. журнал lab_1_2_results.log).

```
# установка / запуск PostgreSQL (Docker)
$ docker compose --env-file /dev/null up -d
[+] Container psiiz-postgres Started
$ docker ps --filter name=psiiz-postgres
NAMES          STATUS
psiiz-postgres Up (healthy)  0.0.0.0:5432->5432/tcp
$ pip install -r requirements.txt
Successfully installed psycopg2-binary...
```

Рисунок 1 – Запуск PostgreSQL в Docker

```
# настройка политики доступа (роли + GRANT)
$ python configure.py
[OK] подключение к PostgreSQL (postgres)
[OK] роль lab_admin (admin)
[OK] роль lab_user (user)
[OK] роль lab_guest (guest)
[OK] база psiiz_demo
[OK] таблицы public_data, confidential_data
[OK] GRANT: lab_user=write public/read secret, lab_guest=read public
[OK] состояние: lab_state.json
```

Рисунок 2 – Создание ролей, базы и GRANT

```
$ python test.py # проверка политики
=== Проверка политики безопасности PostgreSQL ===
[TEST] admin: чтение public_data | OK
[TEST] admin: чтение confidential_data | OK
[TEST] admin: запись public_data | OK
[TEST] admin: запись confidential_data | OK
[TEST] admin: CREATE TABLE | OK
[TEST] user: чтение public_data | OK
[TEST] user: запись public_data | OK
[TEST] user: чтение confidential_data | OK
[TEST] user: запись confidential_data запрещено | OK | permission denied for table confidential_data
[TEST] user: CREATE TABLE запрещено | OK | permission denied for schema public
LINE 1: CREATE TABLE user_test (id int)
          ^
[TEST] guest: чтение public_data | OK
[TEST] guest: запись public_data запрещено | OK | permission denied for table public_data
[TEST] guest: чтение confidential_data запрещено | OK | permission denied for table confidential_data
=== Готово, лог: /Users/dmitry/Desktop/BSUIR/PSIIZ/sem_7/lab_1.2/lab_1_2_results.log ===
```

Рисунок 3 – Результаты проверки политики доступа

```
# удаление БД, ролей и контейнера
$ python cleanup.py
[OK] база psiiz_demo удалена
[OK] роль lab_admin удалена
[OK] роль lab_user удалена
[OK] роль lab_guest удалена
[OK] удалён lab_state.json
$ docker compose --env-file /dev/null down -v
[+] Volume lab_12_pg_data Removed
```

Рисунок 4 – Удаление базы, ролей и контейнера

Фрагмент проверок для роли user (запись в confidential запрещена):

```
ok, err = try_sql("lab_user", pw, db, "SELECT *FROM confidential_data")
result("user: чтение confidential_data", ok, err)

ok, err = try_sql("lab_user", pw, db, "INSERT INTO confidential_data (title,
body) VALUES ('x','x')")
result("user: запись confidential_data запрещена", not ok, err if not ok else
"операция прошла")

ok, err = try_sql("lab_user", pw, db, "CREATE TABLE user_test (id int)")
result("user: CREATE TABLE запрещено", not ok, err if not ok else "операция
прошла")
```

4 ВЫВОД

Семантическая составляющая. На примере PostgreSQL показано, как принцип наименьших привилегий реализуется ролями СУБД: администратор управляет всеми объектами, пользователь работает в рамках выданных прав, гость ограничен чтением публичных данных.

Прагматическая составляющая. Подготовлен набор скриптов install → configure → test → cleanup, позволяющий воспроизводимо демонстрировать политику доступа и сравнивать её с разграничением прав на уровне ОС (лабораторная работа № 1, часть 1).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ховард, М. Защищённый код / М. Ховард, Д. Лебланк. – М. : Русская редакция, 2004. – 704 с.

2. PostgreSQL Documentation. Privileges [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/current/ddl-privs.html>.
3. CWE – Common Weakness Enumeration [Электронный ресурс]. – Режим доступа: <https://cwe.mitre.org/>.